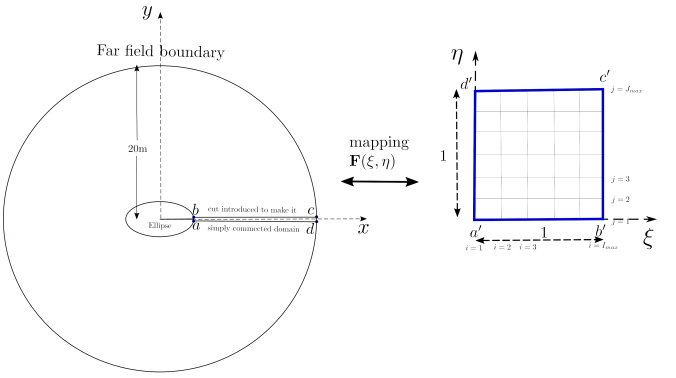
CFD Assignment 2

Ambuj Nayan
24M0003
Aerospace Engineering
Indian Institute of Technology Bombay

I. Problem Statement

Consider a physical domain between a ellipse (with semi-major axis a 1 m and semi-minor axis 0.5 m) and far field boundary located at 20m as shown in the Figure 1. A structured computational grid of size 101×81 is to be generated, where there are 101 points in the ξ-direction and 81 points in the η-direction.



(i) Physical Domain (ii) Computational Domain

Figure 1: Problem Statement

The physical grid is obtained using the **Transfinite Interpolation (TFI)** method, ensuring smooth mapping between computational coordinates (ξ, η) and the physical domain (x, y).

The computational coordinates are defined as follows:

- $\xi \in [0, 1]$ represents the angular distribution along the boundary of the ellipse and the far-field circle.
- $\eta \in [0, 1]$ controls the radial stretching, transitioning from the elliptical inner boundary to the circular outer boundary.

II. Implementation

A. Derivations

The mapping function $\mathbf{F}(\xi, \eta) = (x(\xi, \eta), y(\xi, \eta))$ is constructed using transfinite interpolation.

Physical domain:

- **Inner boundary (ellipse):**

$$x_{\text{inner}} = a\cos\theta, \quad y_{\text{inner}} = b\sin\theta, \quad \theta \in [0, 2\pi]$$

- **Outer boundary (circle):**

$$x_{\text{outer}} = R\cos\theta, \quad y_{\text{outer}} = R\sin\theta, \quad \theta \in [0, 2\pi]$$

$$\mathbf{F}(\xi, \eta) = \begin{bmatrix} x(\xi, \eta) \\ y(\xi, \eta) \end{bmatrix}$$

Computational Domain:

- **Corner Points:**

$$\mathbf{F}(0,0) = \begin{bmatrix} a \\ 0 \end{bmatrix}, \quad \mathbf{F}(1,0) = \begin{bmatrix} a \\ 0 \end{bmatrix},$$
$$\mathbf{F}(0,1) = \begin{bmatrix} R \\ 0 \end{bmatrix}, \quad \mathbf{F}(1,1) = \begin{bmatrix} R \\ 0 \end{bmatrix}$$

- **Edges:**

$$\mathbf{F}(\xi, 0) = \begin{bmatrix} a\cos(2\pi\xi) \\ b\sin(2\pi\xi) \end{bmatrix}, \quad \mathbf{F}(0, \eta) = \begin{bmatrix} (1-\eta)a + \eta R \\ 0 \end{bmatrix}$$

$$\mathbf{F}(\xi, 1) = \begin{bmatrix} R\cos(2\pi\xi) \\ R\sin(2\pi\xi) \end{bmatrix}, \quad \mathbf{F}(1, \eta) = \begin{bmatrix} (1-\eta)a + \eta R \\ 0 \end{bmatrix}$$

Using TFI, the mapping function is expressed as:[1]

$$\begin{aligned} P_\xi[\mathbf{F}(\eta)] &= (1-\xi)\mathbf{F}(0,\eta) + \xi\mathbf{F}(1,\eta) \\ &= (1-\xi)\begin{bmatrix} (1-\eta)a + \eta R \\ 0 \end{bmatrix} + \xi\begin{bmatrix} (1-\eta)a + \eta R \\ 0 \end{bmatrix} \end{aligned}$$

$$\begin{aligned} P_\eta[\mathbf{F}(\xi)] &= (1-\eta)\mathbf{F}(\xi,0) + \eta\mathbf{F}(\xi,1) \\ &= (1-\eta)\begin{bmatrix} a\cos(2\pi\xi) \\ b\sin(2\pi\xi) \end{bmatrix} + \eta\begin{bmatrix} R\cos(2\pi\xi) \\ R\sin(2\pi\xi) \end{bmatrix} \end{aligned}$$

$$\begin{aligned} P_\xi P_\eta(\mathbf{F}) &= (1-\xi)(1-\eta)\mathbf{F}(0,0) + (1-\xi)\eta\mathbf{F}(0,1) \\ &\quad + \xi(1-\eta)\mathbf{F}(1,0) + \xi\eta\mathbf{F}(1,1) \\ &= (1-\xi)(1-\eta)\begin{bmatrix} a \\ 0 \end{bmatrix} + (1-\xi)\eta\begin{bmatrix} R \\ 0 \end{bmatrix} \\ &\quad + \xi(1-\eta)\begin{bmatrix} a \\ 0 \end{bmatrix} + \xi\eta\begin{bmatrix} R \\ 0 \end{bmatrix} \end{aligned}$$

TFI Expression:[1]

$$\boxed{\begin{aligned} \mathbf{F}(\xi,\eta) &= P_\xi[\mathbf{F}(\eta)] \oplus P_\eta[\mathbf{F}(\xi)] \\ &= P_\xi[\mathbf{F}(\eta)] + P_\eta[\mathbf{F}(\xi)] - P_\xi P_\eta(\mathbf{F}) \end{aligned}}$$

Substituting we get:

$$\begin{aligned}\mathbf{F}(\xi,\eta) = (1-\xi)\begin{bmatrix}(1-\eta)a+\eta R\\0\end{bmatrix} + \xi\begin{bmatrix}(1-\eta)a+\eta R\\0\end{bmatrix}\\ +(1-\eta)\begin{bmatrix}a\cos(2\pi\xi)\\b\sin(2\pi\xi)\end{bmatrix} + \eta\begin{bmatrix}R\cos(2\pi\xi)\\R\sin(2\pi\xi)\end{bmatrix}\\ -\left\{(1-\xi)(1-\eta)\begin{bmatrix}a\\0\end{bmatrix} + (1-\xi)\eta\begin{bmatrix}R\\0\end{bmatrix}\right.\\ \left.+\xi(1-\eta)\begin{bmatrix}a\\0\end{bmatrix} + \xi\eta\begin{bmatrix}R\\0\end{bmatrix}\right\}\end{aligned}$$

Simplyfying we get:

$$\mathbf{F}(\xi,\eta) = \begin{bmatrix}x(\xi,\eta)\\y(\xi,\eta)\end{bmatrix} = \begin{bmatrix}\cos(2\pi\xi) & 0\\0 & \sin(2\pi\xi)\end{bmatrix}\begin{bmatrix}(1-\eta)a+\eta R\\(1-\eta)b+\eta R\end{bmatrix}$$

$$\begin{aligned}\mathbf{x}(\xi,\eta) &= \cos(2\pi\xi)[(1-\eta)a+\eta R]\\ \mathbf{y}(\xi,\eta) &= \sin(2\pi\xi)[(1-\eta)b+\eta R]\end{aligned}$$

B. Coding

The Python script generates a structured computational grid using the **Transfinite Interpolation (TFI)** method. This approach ensures a smooth mapping between computational coordinates (ξ,η) and physical domain coordinates (x,y). The implementation follows the following sequence of steps, outlined below:

1) **Defining the Grid Resolution:** The number of grid points in the ξ and η directions is set to 101 and 81, respectively.
2) **Defining Boundaries:** The semi-major and semi-minor axes of the ellipse (inner boundary) and the radius of the outer circular boundary are specified.
3) **Discretization of Computational Coordinates:** The computational coordinates ξ and η are evenly spaced in the $[0,1]$ range using `numpy.linspace`.
4) **Initializing the Grid Arrays:** Two 2D arrays, `x` and `y`, are initialized to store the physical coordinates of the grid points.
5) **Mapping from Computational to Physical Space:**
   - The x and y coordinates of each grid point are computed using the TFI method.
   - The radial stretching (η) interpolates between the inner elliptical boundary and the outer circular boundary.
   - The angular distribution (ξ) ensures a smooth transition along the elliptical and circular boundaries.
6) **Grid Visualization:**
   - Blue lines represent constant ξ values (grid lines in the η direction).
   - Red lines represent constant η values (grid lines in the ξ direction).
   - An imaginary cut line at $\theta = 0$ is marked with a dashed black line.
   - Four key points (a, b, c, d) are highlighted on the inner and outer boundaries.
7) **Plot Formatting:** The grid is displayed using Matplotlib, with appropriate labels, equal aspect ratio, and grid styling.

The final output is a structured computational grid, ensuring a smooth mapping from computational space (ξ,η) to physical space (x,y). The generated grid is saved as an image file `generated_grid.png`.

III. Result

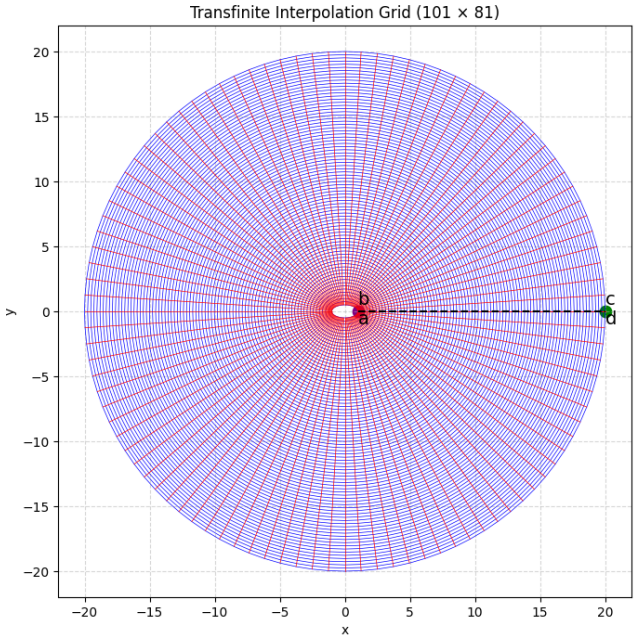


Figure 2: Generated Grid

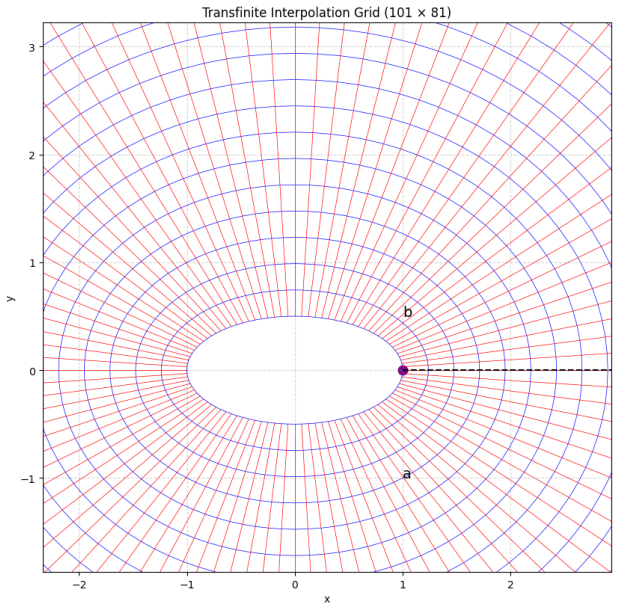


Figure 3: Zoomed View of the grid

APPENDIX A
CODE

A. Code for grid generation

```python
import matplotlib.pyplot as plt
import numpy as np

# Define the grid size
num_xi = 101  # Number of points in the xi-direction
num_eta = 81  # Number of points in the eta-direction

# Define the semi-major and semi-minor axes for the ellipse (inner boundary)
a = 1.0
b = 0.5

# Define the radius for the circle (outer boundary)
R_outer = 20.0

# Generate xi and eta values
xi = np.linspace(0, 1, num_xi)
eta = np.linspace(0, 1, num_eta)

# Initialize x and y grid arrays
x = np.zeros((num_eta, num_xi))
y = np.zeros((num_eta, num_xi))

# Compute the grid points using TFI
for i in range(num_eta):
    for j in range(num_xi):
        x[i, j] = np.cos(2 * np.pi * xi[j]) * ((1 - eta[i]) * a + eta[i] * R_outer)
        y[i, j] = np.sin(2 * np.pi * xi[j]) * ((1 - eta[i]) * b + eta[i] * R_outer)

# Plot the grid
plt.figure(figsize=(10, 8))
for i in range(num_eta):
    plt.plot(x[i, :], y[i, :], "b-", linewidth=0.5)  # Lines along xi
for j in range(num_xi):
    plt.plot(x[:, j], y[:, j], "r-", linewidth=0.5)  # Lines along eta

# Add imaginary cut line (dashed line along theta = 0)
plt.plot(
    [x[0, 0], x[-1, 0]],
    [y[0, 0], y[-1, 0]],
    "k--",
    linewidth=1.5,
    label="Imaginary Cut (θ = 0)",
    )

# Highlight Point a and b
plt.scatter(x[0, 0], y[0, 0], color="purple", s=80, label="Point a/b")
plt.text(x[0, 0], y[0, 0] + 0.5, "b", fontsize=14, color="black")
plt.text(x[0, 0], y[0, 0] - 1.0, "a", fontsize=14, color="black")

# Highlight Point c and d
plt.scatter(x[-1, 0], y[-1, 0], color="green", s=80, label="Point d/c")
plt.text(x[-1, 0], y[-1, 0] + 0.5, "c", fontsize=14, color="black")
plt.text(x[-1, 0], y[-1, 0] - 1.0, "d", fontsize=14, color="black")

```

```
plt.gca().set_aspect("equal")
plt.title("Transfinite Interpolation Grid (101 x 81)")
plt.xlabel("x")
plt.ylabel("y")
plt.grid(True, linestyle="--", alpha=0.5)
plt.savefig("generated_grid.png")
plt.show()
```

Appendix B
Source

References

[1] T. J. Chung, *Computational Fluid Dynamics*. Cambridge University Press, 2010.